

Rapport de Projet

Environnement de Drone pour l'Apprentissage par Renforcement

INFO3 – Méthodes Numériques pour l'Apprentissage Machine

Année 2025/2026

1 Introduction

Ce projet consiste à implémenter un environnement de simulation destiné à l'apprentissage par renforcement (RL). L'environnement modélise un drone autonome évoluant dans une grille 2D, avec des contraintes de batterie, de sécurité et de couverture de zones.

L'objectif principal est de fournir un environnement conforme à une spécification formelle, permettant ultérieurement l'intégration d'un agent RL.

2 Modélisation de l'environnement

2.1 Grille et zones

L'environnement est représenté par une grille discrète de taille $(maxX, maxY)$. Chaque cellule appartient à une zone définie par une fonction déterministe :

- 4 zones sont définies ;
- la division est basée sur les médianes de la grille ;
- chaque cellule appartient à une unique zone.

Cette abstraction permet de raisonner en termes de couverture plutôt qu'en termes de positions individuelles.

2.2 Gestion des dangers

Un ensemble **Danger** contient les cellules dangereuses. Si le drone entre dans une de ces cellules :

- la distance au danger devient 0 ;
- un crash peut être déclenché.

Ce choix simplifie la modélisation tout en conservant un signal critique pour l'agent.

2.3 État du système

L'état est encapsulé dans une classe **State**, contenant :

- position du drone ;
- position de la base ;
- batterie ;
- état opérationnel (airborne, docked, crashed) ;
- zones visitées.

L'état est immuable via copie, ce qui facilite l'exploration exhaustive.

3 Respect des invariants

Une fonction `check_invariant` garantit :

- validité des positions ;
- cohérence des états (ex : pas airborne et docked simultanément) ;
- validité des zones visitées ;
- respect des contraintes batterie.

Toute transition viole l'invariant déclenche une exception, garantissant la robustesse du système.

4 Implémentation des transitions

Chaque action est implémentée comme une fonction retournant une liste d'états possibles :

4.1 Actions déterministes

- `takeoff_cmd`
- `return_home`
- `dock_cmd`
- `undock_cmd`
- `charge_step`
- `emergency_stop`

4.2 Actions non déterministes

- `move_step`
- `avoid_maneuver`

Ces actions retournent plusieurs successeurs, modélisant les choix possibles.

4.3 Gestion du non-déterminisme

- Mode interactif / automatique : un successeur est choisi aléatoirement ;
- Mode exhaustif : tous les successeurs sont explorés.

5 Gestion de la batterie

La batterie est une ressource critique :

- déplacement : coût 1 ;
- évitement : coût 2 ;
- recharge : +5 ;

Ce modèle introduit un compromis entre exploration et sécurité.

6 Modes d'exécution

6.1 Mode interactif

Ce mode permet à l'utilisateur de piloter le drone :

- choix manuel des actions ;
- visualisation de la grille ;
- compréhension du comportement.

Les actions non déterministes sont résolues aléatoirement.

6.2 Mode automatique

Ce mode exécute une séquence d’actions depuis un fichier CSV :

- lecture ligne par ligne ;
- arrêt en cas d’échec ou crash ;
- affichage dynamique de l’état.

Il permet de tester des stratégies reproductibles.

6.3 Mode exhaustif

Ce mode explore l’espace des états via un DFS :

- évite les cycles via un ensemble de visités ;
- gère des limites de profondeur et largeur ;
- construit un graphe de transitions.

Il permet :

- validation formelle ;
- analyse de couverture ;
- étude du non-déterminisme.

7 Exploration récursive

L’algorithme utilisé est une exploration DFS :

- arrêt si état déjà visité ;
- arrêt si profondeur maximale atteinte ;
- exploration limitée en largeur si nécessaire.

Cela garantit que seuls les états atteignables sont explorés.

8 Choix d’implémentation

8.1 Architecture

Le projet est structuré en modules :

- **Environment** : logique principale ;
- **State** : représentation des états ;
- **Grid** : gestion de la grille ;
- **Modes** : interaction utilisateur.

8.2 Séparation logique

Chaque responsabilité est isolée :

- transitions dans **Environment** ;
- affichage dans **main** ;
- exploration dans **ExhaustiveMode**.

8.3 Robustesse

- vérification systématique des invariants ;
- gestion explicite des erreurs ;
- encapsulation des transitions.

9 Limites et améliorations

9.1 Limites

- pas d’algorithme RL implémenté ;
- représentation simple du danger ;
- absence de visualisation graphique avancée.

9.2 Améliorations possibles

- ajout d’un agent RL (Q-learning, DQN) ;
- interface graphique ;
- amélioration du modèle de perception du danger.

10 Conclusion

Ce projet permet de construire un environnement RL rigoureux, respectant une spécification formelle et intégrant :

- gestion des contraintes ;
- non-déterminisme ;
- exploration d’espace d’états.

Il constitue une base solide pour l’expérimentation d’algorithmes d’apprentissage par renforcement.